

KS

KaushalSetu API

Backend documentation - 2026-08-29

Backend Documentation

Verified Skills. Trusted Workers. Faster Hiring.

Technical documentation for the Node.js, Express, MongoDB, and Socket.IO backend powering KaushalSetu.

Generated from README.md and files under server/src, server/tests, and server/package.json.

7

Main API domains mounted under /api

11

Mongoose models registered at startup

JWT

Stateless access and refresh token auth

Scope: backend architecture, setup, configuration, data model, endpoints, workflows, security, realtime events, algorithms, and test coverage notes.

Contents

1. Backend Summary

2. Architecture

3. Runtime And Configuration

4. Folder Structure

5. API Reference

6. Data Model
7. Core Workflows

8. Algorithms

9. Realtime Messaging

10. Security And Validation

11. Testing And Operations

Backend Summary

KaushalSetu is a recruitment platform for India's blue-collar workforce. The backend owns user identity, worker and employer profiles, skill verification, job discovery, applications, hiring pipeline management, messaging, notifications, and admin analytics.

RUNTIME

Node.js + Express 5

ES modules, JSON APIs, centralized route mounting.

PERSISTENCE

MongoDB + Mongoose

Document schemas, embedded profile records, refs, indexes, geospatial fields.

REALTIME

Socket.IO

Authenticated sockets and user-specific rooms for message events.

API base: all resource routes mount below `/api`. Health checks are available at `/` and `/health` outside the API prefix.

Express 5

Mongoose 9

JWT

bcryptjs

Zod

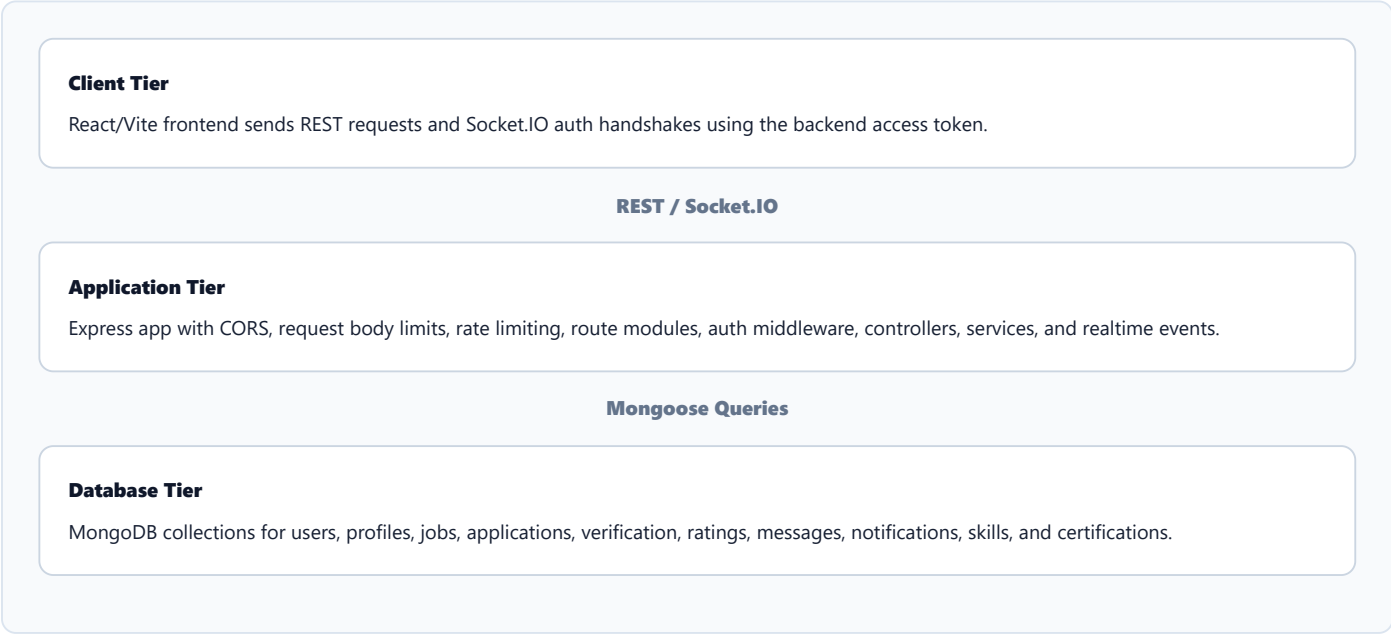
express-rate-limit

Socket.IO

dotenv

Architecture

The backend follows the README's layered structure: routes map HTTP endpoints, controllers shape request/response behavior, services hold reusable business logic, models define persistence, middleware guards and validates requests, and utilities centralize cross-cutting helpers.



Request Lifecycle



The app disables `x-powered-by`, accepts JSON and URL-encoded bodies up to `1mb`, applies a global API limiter to `/api`, and ends with not-found and centralized error middleware.

Runtime And Configuration

INSTALL

```
cd server
npm install
```

RUN

```
npm run dev
npm start
npm run seed
```

VARIABLE	DEFAULT / EXAMPLE	PURPOSE
NODE_ENV	development	Controls environment behavior and stack traces.
PORT	3000	HTTP port used by the Express server.
CORS_ORIGIN	http://localhost:5173	Comma-separated allow-list for REST and Socket.IO origins.
MONGODB_URI	mongodb://127.0.0.1:27017/kaushalsetu	MongoDB connection string.
JWT_SECRET	replace with production secret	Signs short-lived access tokens.
JWT_EXPIRES_IN	7d	Access token lifetime.
REFRESH_TOKEN_SECRET	replace with different production secret	Signs refresh tokens.
REFRESH_TOKEN_EXPIRES_IN	30d	Refresh token lifetime.

Production note: replace default JWT secrets with long random values, set a production CORS allow-list, and use a managed MongoDB URI.

Folder Structure

AREA	PATH	RESPONSIBILITY
Entry point	<code>src/server.js</code>	Creates the Express app, configures CORS/body parsing, mounts /api, exposes / and /health, and starts Socket.IO.
Routes	<code>src/routes/*.js</code>	Maps public and protected HTTP paths to controllers.
Controllers	<code>src/controllers/*.js</code>	Handle request/response concerns, status codes, simple query composition, and workflow orchestration.
Services	<code>src/services/*.js</code>	Hold reusable business logic for auth, profile updates, verification, messaging, matching, scoring, and notifications.
Models	<code>src/models/*.js</code>	Mongoose schemas, embedded profile subdocuments, indexes, and collection definitions.
Middleware	<code>src/middleware/*.js</code>	Authentication, role guards, validation, rate limiting, not-found, and error handling.
Utilities	<code>src/utils/*.js</code>	JWT helpers, password hashing, validation schemas, score and match algorithms, constants, and structured logging.
Seed data	<code>src/seed/*.js</code>	Demo users, workers, employers, jobs, applications, messages, and supporting data.
Tests	<code>tests/*.test.js</code>	Coverage targets for auth, applications, ownership, permissions, jobs, messages, and verification.

API Reference

Every route returns JSON shaped around `success`, optional `message`, and `data` or `errors`. Protected routes expect `Authorization: Bearer <token>`.

System And Auth

METHOD	PATH	ACCESS	PURPOSE
GET	/	Public	Root status response: returns success and running message.
GET	/health	Public	Reports API health and Mongoose connection state.
POST	/api/auth/register	Public, auth limiter	Creates a worker, employer, or admin user and returns access/refresh tokens.
POST	/api/auth/login	Public, auth limiter	Validates credentials, returns a session and role profile.
POST	/api/auth/refresh	Public, auth limiter	Rotates access and refresh tokens using the refresh token secret.
POST	/api/auth/logout	Public	Stateless logout endpoint; client clears tokens.

Worker Profiles And Jobs

METHOD	PATH	ACCESS	PURPOSE
GET	/api/workers/me	WORKER	Returns the authenticated worker profile.
GET	/api/workers/:id	Public	Returns a worker profile by profile id or user id.
GET	/api/workers/:id/profile	Public	Alias for public worker profile lookup.
PUT	/api/workers/:id	WORKER	Updates profile fields and refreshes the trust score.
GET	/api/workers/:id/skills	Public	Lists skill entries embedded on the worker profile.
POST	/api/workers/:id/skills	WORKER	Adds a self-declared or supplied skill; requires name.
PUT	/api/workers/:id/skills/:skillId	WORKER	Updates a profile skill and recalculates trust score.
DELETE	/api/workers/:id/skills/:skillId	WORKER	Removes a profile skill and recalculates trust score.
GET	/api/workers/:id/certifications	Public	Lists embedded certification records.
POST	/api/workers/:id/certifications	WORKER	Adds a certification; requires name and issuer.
GET	/api/workers/:id/work-history	Public	Lists embedded work history entries.
POST	/api/workers/:id/work-history	WORKER	Adds a work history entry and refreshes scoring.
GET	/api/workers/:id/trust-score	Public	Returns score and component breakdown.
GET	/api/workers/:id/applications	WORKER or ADMIN	Lists applications for a worker.
GET	/api/jobs/recommended	WORKER	Returns open jobs ordered by controller-level skill match score.
GET	/api/jobs/search	Public	Searches open jobs by text, category, salary, and employment type.
GET	/api/jobs/:id	Public	Returns one job by id.
POST	/api/applications	WORKER	Creates an application; requires jobId.

Employer Hiring Surface

METHOD	PATH	ACCESS	PURPOSE
GET	/api/employers/me	EMPLOYER	Returns the authenticated employer profile.
PUT	/api/employers/:id	EMPLOYER	Updates employer profile details.
GET	/api/employers/:id/jobs	EMPLOYER	Lists jobs created for that employer id.
POST	/api/employers/:id/jobs	EMPLOYER	Creates a job; route validation requires title.
PUT	/api/employers/:id/jobs/:jobId	EMPLOYER	Updates an existing job document.
DELETE	/api/employers/:id/jobs/:jobId	EMPLOYER	Soft deletes by changing job status to CLOSED.
GET	/api/employers/:id/candidates	EMPLOYER	Lists all worker profiles, including linked user metadata.
GET	/api/employers/:id/candidates/search	EMPLOYER	Filters workers by skill, occupation, experience, availability, or salary.
GET	/api/employers/:id/candidates/:workerId/details	EMPLOYER	Gets a candidate profile by profile id or user id.
POST	/api/employers/:id/candidates/:workerId/shortlist	EMPLOYER	Creates or updates an application as SHORTLISTED; requires jobId.
GET	/api/employers/:id/pipeline	EMPLOYER	Groups applications into APPLIED, SHORTLISTED, INTERVIEW, HIRED, REJECTED.
PUT	/api/employers/:id/pipeline/:appId/status	EMPLOYER	Moves an application stage; records interview/hire timestamps.
GET	/api/employers/:id/analytics	EMPLOYER	Returns job, applicant, interview, hire, and conversion counts.

Verification, Admin, And Messaging

METHOD	PATH	ACCESS	PURPOSE
POST	/api/verifications/request	WORKER	Creates a verification request; requires workerId and skillName.
GET	/api/verifications/:workerId	Authenticated	Lists verification records for one worker.
GET	/api/admin/users	ADMIN	Lists users without passwords.
PUT	/api/admin/users/:id/status	ADMIN	Updates the user verified flag.
GET	/api/admin/workers	ADMIN	Lists worker profiles with linked user metadata.
GET	/api/admin/employers	ADMIN	Lists employer profiles with linked user metadata.
GET	/api/admin/jobs	ADMIN	Lists all jobs with employer email.
GET	/api/admin/analytics	ADMIN	Returns platform-level counts and verification totals.
GET	/api/admin/platform-stats	ADMIN	Returns dashboard summary counts.
GET	/api/admin/verifications	ADMIN	Lists pending verification records.
PUT	/api/admin/verifications/:id/approve	ADMIN	Approves verification, updates profile skill, notifies worker.
PUT	/api/admin/verifications/:id/reject	ADMIN	Rejects verification, updates profile skill, notifies worker.
GET	/api/messages/conversations	Authenticated	Returns conversations, unread counts, partner display name, and last message.

METHOD	PATH	ACCESS	PURPOSE
GET	/api/messages/recipients	Authenticated	Searches opposite-role recipients; accepts q and returns up to 25.
POST	/api/messages	Authenticated	Saves a message; requires recipientId and content.
GET	/api/messages/:otherUserId	Authenticated	Lists messages in a two-user conversation.
POST	/api/messages/:otherUserId/read	Authenticated	Marks unread incoming messages as read.

Data Model

The backend uses Mongoose models and a mix of referenced collections and embedded profile subdocuments. Worker skills, certifications, and work history are embedded for fast profile rendering; jobs, applications, ratings, verification records, messages, and notifications live as separate documents.

MODEL	KEY FIELDS	INDEXES / CONSTRAINTS	BACKEND ROLE
User	email, password, role, verified	email unique/indexed; role enum WORKER, EMPLOYER, ADMIN	Authentication and role identity.
WorkerProfile	userId, name, phone, location, skills, certifications, workHistory, salary, availability, score fields	userId unique/indexed; location 2dsphere; primaryOccupation indexed	Portable worker identity, profile data, reputation, and score storage.
EmployerProfile	userId, companyName, industry, location, verified, registrationNumber	userId unique/indexed; companyName indexed; registrationNumber unique sparse	Employer/company identity and verification metadata.
Job	employerId, title, category, requiredSkills, experience, salary, location, status	employerId, title, category, status indexed; location 2dsphere	Open, closed, or filled hiring demand.
Application	jobId, workerId, status, matchScore, appliedAt, stage timestamps, notes	jobId/workerId/status indexed; unique jobId+workerId pair	Worker interest and employer pipeline state.
Skill	name, category, description, totalWorkers	name unique/indexed; category indexed	Canonical skill catalog.
Certification	name, issuer, category, validityYears	name unique	Reusable certification definitions.
Verification	workerId, skillName, skillId, status, type, verifier, documentUrl, notes	workerId and verificationStatus indexed	Skill verification request and review record.
Rating	workerId, employerId, jobId, rating, feedback, category	workerId/employerId/jobId indexed; unique workerId+jobId+category	Employer feedback signal for reputation.
Message	senderId, recipientId, jobId, content, read, readAt	senderId/recipientId indexed; compound conversation index	Persistent direct messaging.
Notification	userId, type, resourceId, title, message, read, readAt	userId indexed	User-facing event feed.

Important Status Enums

User roles: WORKER, EMPLOYER, ADMIN

Application: APPLIED, SHORTLISTED, INTERVIEW, REJECTED, HIRED

Verification: PENDING, APPROVED, REJECTED

Availability: AVAILABLE, UNAVAILABLE, PART_TIME

Job: OPEN, CLOSED, FILLED

Core Workflows

1. Registration

Zod validates email, role, password length, and Indian phone number shape. Passwords are hashed with bcryptjs. Worker and employer roles get profile documents immediately.

2. Login And Refresh

Login returns a user summary, an access token, a refresh token, and the relevant role profile. Refresh verifies the refresh token and rotates both tokens.

3. Worker Profile Growth

Skills, certifications, work history, ratings, completed jobs, and verification state feed the Kaushal Trust Score. Profile updates call scoring refresh where implemented.

4. Employer Hiring

Employers post jobs, browse/search candidates, shortlist workers into applications, group applications into pipeline stages, and update stage status.

5. Verification Review

Workers request verification for a skill. Admin review updates the verification record, mirrors status onto the worker skill, refreshes score, and creates a notification.

6. Messaging

Authenticated users message opposite-role recipients. REST endpoints persist messages and Socket.IO emits realtime events to user-specific rooms.

Hiring Pipeline Shape

```
{
  APPLIED: [...],
  SHORTLISTED: [...],
  INTERVIEW: [...],
  HIRED: [...],
  REJECTED: [...]
}
```

When an application is moved to INTERVIEW, `movedToInterviewAt` is set. When moved to HIRED, `hiredAt` is set and the worker profile's `completedJobsCount` is incremented.

Algorithms

Kaushal Trust Score

The trust score is deterministic, capped at 100, and stored on `WorkerProfile.kaushalTrustScore` with a `trustScoreBreakdown` object. Scoring uses verified skill state, experience, ratings, completed jobs, certifications, assessment status, and profile completeness.

FACTOR	MAX POINTS	HOW IT IS CALCULATED
Verified skills	25	0, 10, 18, or 25 points based on approved/self-verified skill count thresholds.
Experience	18	0 to 18 points based on years of experience, from partial experience through 10+ years.
Employer ratings	20	0 to 20 points from average rating bands starting at 3.0 stars.
Completed jobs	14	0 to 14 points based on completedJobsCount.
Certifications	10	0, 3, 6, or 10 points from certification count.
Assessment pass	8	8 points if assessmentPassed is true.
Profile completeness	5	One point each for photo, skills, work history, certifications, and languages.

Candidate / Job Matching

The reusable matching service calculates five components, rounds their average, and returns both a score and breakdown. It compares required skills against worker skills, maps availability to 100/60/0, floors salary alignment at 20, gives experience either 100 or 40, and includes the worker trust score.

```
score = round((skillMatch + availability + salaryAlignment + experience + trustScore) / 5)
```

Implementation note: `/api/jobs/recommended` currently performs a simpler controller-level skill match for open jobs. The richer matching service is available for shared ranking logic.

Realtime Messaging

Socket.IO is attached to the HTTP server during startup. Each socket must provide an access token in `handshake.auth.token`. After verification, the socket joins a room named `user:<userId>` so events can target a specific user.

EVENT

`message:new`

Emitted to sender and recipient after a message is persisted.

EVENT

`message:read`

Emitted to the other user when a thread is marked read.

Notification creation for messages is best-effort; a notification failure does not fail message sending.

Security And Validation

Authentication

Bearer tokens are verified with the access token secret. Refresh tokens use a separate secret and lifetime. The request identity is normalized to `{ id, role }`.

Authorization

`authorize(...roles)` enforces role-based access for worker, employer, and admin routes. Tests also describe ownership-boundary expectations that should be preserved as routes evolve.

Validation

Registration uses Zod. Required body fields are checked by middleware. Phone validation accepts Indian numbers with optional `+91` and mobile prefixes 6-9.

Abuse Protection

The global API limiter permits 120 requests per minute. Auth endpoints have a stricter 20 attempts per 15 minutes window.

Password Storage

Passwords are salted and hashed with bcryptjs using 10 rounds. Password fields are excluded from admin user listings.

Error Handling

Centralized middleware maps cast errors to 400, validation errors to 400, duplicate keys to 409, and includes stack traces only in development.

Testing And Operations

The server contains focused test files for major backend contracts. They are useful as living documentation of expected behavior and should be wired to a real test runner command in `server/package.json` if not already handled externally.

TEST FILE	COVERAGE INTENT
auth.test.js	Registration/login paths, duplicate users, invalid roles, phone validation, auth route limiting.
permissions.test.js	Missing/invalid tokens, role guard behavior, non-admin admin access, bad credentials.
ownership.test.js	Expected safety checks for profile/application/job ownership boundaries.
jobs.test.js	Employer job creation, listing, update, soft delete, public search, missing job handling.
applications.test.js	Application creation and identity spoofing expectations.
messages.test.js	Recipient search by opposite role and creating a first conversation.
verification.test.js	Worker requests, admin pending queue, approvals, and skill status propagation.

Operational Checklist

Before local demo

Install server packages, create `.env`, start MongoDB, run `npm run seed`, then `npm run dev`.

Before deployment

Set secrets, restrict CORS, configure MongoDB Atlas or equivalent, seed only appropriate demo data, and monitor health at `/health`.